

2025

MASTER'S THESIS

**A Deep Reinforcement Learning  
Approach to Roundabout Traffic Signal  
Control**

ACADEMIC SUPERVISOR: Kryssanov Victor

Graduate School of Information Science and Engineering Ritsumeikan  
University

MASTER'S PROGRAM  
MAJOR in Advanced Information Science and Engineering

STUDENT ID: 6613230014-4

NAME: CHEN Cheng-Yu

# Abstract

Urban traffic congestion is a significant challenge for modern cities. This problem leads to increased travel times, energy consumption, and environmental pollution. While conventional traffic signal control methods, such as fixed-time or rule-based strategies, are still widely used, they often fail to adapt to complex traffic dynamics, especially in urban roundabouts. This study proposes a deep reinforcement learning (DRL) framework for intelligent traffic signal control. In this research, I specifically train and test the model in the Gongguan Roundabout in Taipei, Taiwan—an intersection with high traffic volumes and asymmetric geometry. A simulation setup is created using the SUMO software, and a Recurrent Proximal Policy Optimization (R-PPO) algorithm with LSTM memory is used to address partial observability and temporal dependencies in the traffic flow. Experiments conducted under different traffic conditions, including peak traffic hours, demand fluctuations, and incident disruptions, demonstrated that the proposed R-PPO framework significantly outperforms baseline strategies. These findings show the effectiveness of memory-enhanced DRL in achieving adaptive, stable, and fair traffic signal control in real-world roundabout scenarios. The source code and experimental scripts could be obtained from [https://github.com/chenyuuuuuu/DRL\\_Gongguan](https://github.com/chenyuuuuuu/DRL_Gongguan)

# Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                              | <b>iv</b> |
| <b>List of Tables</b>                                               | <b>v</b>  |
| <b>1 Introduction</b>                                               | <b>1</b>  |
| 1.1 Background . . . . .                                            | 1         |
| 1.2 Problem Statement . . . . .                                     | 2         |
| 1.3 Research Objectives . . . . .                                   | 2         |
| 1.4 Thesis Organization . . . . .                                   | 4         |
| <b>2 Related Work</b>                                               | <b>5</b>  |
| 2.1 AI-Based Methods in Traffic Signal Control . . . . .            | 5         |
| 2.2 Deep Reinforcement Learning Methods . . . . .                   | 6         |
| 2.2.1 Deep Q-Networks and Value-Based Deep RL . . . . .             | 6         |
| 2.2.2 Actor-Critic and Policy Gradient Methods . . . . .            | 6         |
| 2.2.3 Recurrent Policies in Traffic Signal Control . . . . .        | 8         |
| <b>3 Proposed Method</b>                                            | <b>9</b>  |
| 3.1 Reinforcement Learning for Traffic Signals . . . . .            | 9         |
| 3.2 System Architecture . . . . .                                   | 10        |
| 3.3 Component Design . . . . .                                      | 12        |
| 3.3.1 Observation Space . . . . .                                   | 12        |
| 3.3.2 Action Space . . . . .                                        | 15        |
| 3.3.3 Reward Design . . . . .                                       | 16        |
| 3.4 Recurrent Proximal Policy Optimization . . . . .                | 18        |
| 3.4.1 Policy Network Architecture . . . . .                         | 18        |
| 3.4.2 Buffer, Advantage Estimation, and Loss Optimization . . . . . | 21        |
| <b>4 Experiments</b>                                                | <b>26</b> |
| 4.1 Simulation Setup . . . . .                                      | 26        |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 4.2      | Traffic Scenarios . . . . .                  | 28        |
| 4.3      | Baselines and Evaluation Metrics . . . . .   | 28        |
| <b>5</b> | <b>Results and Analysis</b>                  | <b>30</b> |
| 5.1      | Training Performance and Stability . . . . . | 30        |
| 5.2      | Generalization Performance . . . . .         | 33        |
| 5.2.1    | High-Demand Scenario . . . . .               | 33        |
| 5.2.2    | Accident Scenario . . . . .                  | 34        |
| 5.3      | Discussion . . . . .                         | 35        |
| <b>6</b> | <b>Conclusions</b>                           | <b>37</b> |
| 6.1      | Summary of Findings . . . . .                | 37        |
| 6.2      | Limitations . . . . .                        | 37        |
| 6.3      | Future Work . . . . .                        | 38        |
|          | <b>References</b>                            | <b>45</b> |

# List of Figures

|    |                                                                                                                       |    |
|----|-----------------------------------------------------------------------------------------------------------------------|----|
| 1  | Conceptual loop of DRL-based traffic signal control. . . . .                                                          | 10 |
| 2  | Proposed system architecture. . . . .                                                                                 | 11 |
| 3  | Index of the target green phase. . . . .                                                                              | 15 |
| 4  | Architecture of the RPPO policy network with a shared encoder,<br>LSTM memory, and actor-critic output heads. . . . . | 18 |
| 5  | The forward and backward flow of hidden states in a 64-step rollout<br>segment during Recurrent PPO training. . . . . | 23 |
| 6  | The real Gongguan Roundabout (a) and its SUMO simulation<br>environment (b). . . . .                                  | 26 |
| 7  | Entry volumes and turning ratios at the Gongguan Roundabout<br>(data source: Taipei City Government). . . . .         | 27 |
| 8  | Training curve of average step reward for RPPO, PPO, and DDQN. . . . .                                                | 30 |
| 9  | Training curve of average vehicle speed for RPPO, PPO, and DDQN. . . . .                                              | 31 |
| 10 | Training curve of average queue length for RPPO, PPO, and DDQN. . . . .                                               | 32 |
| 11 | Explained variance of value function prediction during training. . . . .                                              | 33 |

# List of Tables

|   |                                                                   |    |
|---|-------------------------------------------------------------------|----|
| 1 | Comparison of Deep RL Methods for Traffic Signal Control. . . . . | 8  |
| 2 | Notations used in the DRL training framework . . . . .            | 12 |
| 3 | Vehicle types, proportion and SUMO configuration parameters. . .  | 28 |
| 4 | Performance under High-Demand Scenario . . . . .                  | 34 |
| 5 | Performance under Accident Scenario . . . . .                     | 34 |

# 1. Introduction

Traffic congestion in many urban areas around the world is becoming more serious due to city populations grow and more people have their own private vehicles. This situation make people spend longer travel time, operating costs increase and air quality also gets worser. Tincani et al. [1] point out that congestion produces higher levels of PM2.5 and ozone ( $O_3$ ), which are harmful to human health. To solve traffic congestion problems, numbers of studies [2, 3] show that good traffic signal control is one important way to ease these problems and improve city mobility.

The rest of this chapter first introduces the background, then explains the problem statement and research objectives.

## 1.1 Background

Traffic signal control is widely used to relieve congestion. The simplest way is fixed-time control. It follows a timetable made from historical data, so implementation is cheap and easy. But it can not react when demand changes suddenly. Actuated control is an improved method. It reads sensor data and can extend or shorten a phase. But it focuses only on single intersection and may not able to coordinate across multiple junctions.

In recent years, new communication technologies like Internet of Things (IoT) have allowed vehicles and roadside units to exchange messages quickly. Vehicular Ad Hoc Networks (VANETs) support applications such as adaptive signal control and traffic prediction [4]. These technologies give a good foundation for intelligent traffic systems that can make better use of modern methods like reinforcement learning (RL) and 5G.

Due to VANETs, many adaptive traffic-control systems (ATCS) have been built, such as InSync [5], RHODES [6], and decentralised cycle-free methods [7]. They adjust signal timing according to current traffic and often work better than fixed-time plans. However, most of them still rely on rules or models designed by experts. When the traffic pattern is very different from the design, their performance drops.

To deal with the limitation of ATCS, reinforcement learning (RL) [8–10] lets an agent learn a control policy by trying different actions and getting feedback from the environment. It does not need a detailed traffic model and can adapt to dynamic or uncertain conditions.

## 1.2 Problem Statement

Although reinforcement learning (RL) has shown some good results in traffic signal control, most experiments are work in simple four-way intersections with balanced flow. Real city intersections are often more complicated like roundabouts. They have a circular layout, many entry lanes, and strong merging pressure. And in many Asian cities, motorcycles account for a large percentage of traffic flow, making traffic even more difficult to predict and control.

To address this research gap, this study focuses on the Gongguan Roundabout in Taipei, Taiwan. It is a complex and heavily used urban intersection. The roundabout features seven entry and exit approaches and handles exceeding 10,000 vehicles during peak hours. Notably, approximately 76% of the total traffic flow is motorcycles, contributing to unstable vehicle interactions and increased merging pressure. Moreover, According to government records, Gongguan has been ranked as the most accident-prone intersection in Taipei for seven consecutive years. From 2017 to 2023, it recorded 474 traffic accidents, it means that there is one incident every 6.2 days [11, 12].

All of the problems in Gongguan make it an ideal but challenging test case for advanced traffic signal control. Existing fixed-time and rule-based systems have poor performance. Therefore, this study aims to explore a more adaptive and intelligent control framework using deep reinforcement learning.

## 1.3 Research Objectives

Urban roundabouts often involve complex traffic behaviors such as asymmetric inflows and continuous merging. These characteristics make it difficult for



traditional traffic signal control methods to perform effectively. The main objective of this study is to develop a deep reinforcement learning (DRL) framework that can adaptively control traffic signals in such environments, aiming to reduce vehicle delay, improve speed, and enhance control stability.

To accomplish this, four technical objectives are proposed:

1. **Construct a realistic simulation environment based on real-world roundabout conditions**

This study models the Gongguan Roundabout in a microscopic traffic simulator—SUMO (Simulation of Urban Mobility) [13] with accurate geometry, lane configurations, and vehicle types including cars, motorcycles and buses. Traffic demand is generated dynamically with controlled variation, allowing the agent to experience diverse traffic conditions during training. This simulation serves as a testbed for training and benchmarking DRL controllers.

2. **Develop a memory-based DRL agent using Recurrent Proximal Policy Optimization model for adaptive roundabout signal control**

Roundabout traffic often changes over time and cannot be fully observed at each moment. To deal with this, the agent is trained with Recurrent Proximal Policy Optimization (R-PPO), which includes an LSTM layer. This helps the agent remember past traffic situations and make better decisions step by step.

3. **Design the observation, action, and reward structures based on the unique characteristics of roundabout traffic, to support R-PPO agent effective learning**

In reinforcement learning, observation, action, and reward are the three core components that define how the agent interacts with the environment. To support learning in a dynamic and partially observable roundabout environment, the observation, action, and reward must reflect both spatial and temporal traffic patterns.

#### **4. Evaluate the proposed control framework under diverse traffic scenarios and compare it with established baselines**

The agent is tested under multiple realistic conditions, including peak-hour congestion, randomly fluctuating demand, and entry imbalance. Performance is benchmarked against fixed-time schedules, rule-based controllers, and non-recurrent DRL models baseline. Evaluation metrics include average travel time, total delay, switch count, fairness index (e.g., Gini coefficient), and phase duration stability. This provides a comprehensive assessment of both performance.

## **1.4 Thesis Organization**

The rest of this thesis is structured as follows. Chapter 2 discusses the evolution of AI methods in traffic signal control, including multiple deep reinforcement learning (DRL) approaches. Chapter 3 introduces the proposed DRL framework, including the problem formulation as a partially observable Markov decision process (POMDP), the simulation-control architecture, the design of the observation space, action space, reward function, and the algorithms. Chapter 4 describes the evaluation setting. This includes the design of baseline strategies, different traffic scenarios, performance metrics, as well as the implementation settings and training configurations. Chapter 5 presents the experimental results. The chapter provides a comparative analysis of the proposed Recurrent PPO framework against baselines, behavioral analysis of learned policies, and evaluation under varying traffic conditions. Finally, Chapter 6 concludes the thesis with a summary of key findings, limitations, and future research directions.

## 2. Related Work

### 2.1 AI-Based Methods in Traffic Signal Control

The earliest applications of AI in traffic light control is the use of **fuzzy logic**, **artificial neural networks**, and **basic reinforcement learning**, each having different advantages.

Fuzzy logic [14] is a powerful tool for processing nondeterministic and non-linear problems. It can represent fuzzy and qualitative knowledge, and so it can imitate human making decisions. But fuzzy systems require rules that are specifically designed by expert and do not improve over time, limiting their adaptability in highly dynamic scenarios. In the large-scale system, there are many interactional factors, and it is difficult to describe the whole system using some qualitative knowledge. It is just the limitations of fuzzy control methods. In other word, it is more appropriate to use fuzzy control methods for traffic signal control of the isolated intersection.

Artificial neural networks (ANNs) support function approximation capabilities, enabling the mapping from traffic states to control actions based on historical data. ANNs [15] can learn complex nonlinear relationships in traffic behavior and were initially applied in supervised learning settings for signal timing prediction or classification. However, these models require extensive labeled datasets and need to be retrain when traffic patterns change.

In Reinforcement learning (RL) there is an agent that learn optimal control strategies through trial-and-error while interacting with the environment. Early work with tabular RL methods, such as Q-learning [16, 17] learns action-value functions and select actions by explicit maximization. It demonstrates that AI agents could achieve adaptive control policies without pre-defined rules or labels.

Despite fuzzy logic, artificial neural networks, and basic reinforcement learning have conceptual advantages, early AI-based methods were still constrained to simplified settings and were seldom applied at scale or in real-world deployments.

## 2.2 Deep Reinforcement Learning Methods

In high-dimensional state spaces, standard RL algorithms can not efficiently compute the value functions or policy functions for all states. Although some linear function approximation methods are proposed for solving the large state space problem in RL, their capabilities still have a limit. However, this problem can be handled by deep learning based function approximators.

### 2.2.1 Deep Q-Networks and Value-Based Deep RL

Traditional Q-learning faces the *curse of dimensionality*, where the Q-table grows rapidly with the number of states and actions. Deep Q-Networks (DQN) [18–21] solve this problem by using deep neural networks to approximate the Q-function. This allows generalization across similar states and removes the need to store every state-action value. To improve stability, DQN introduces two main techniques. First, a separate target network is used to compute TD targets, reducing feedback instability. Second, an experience replay buffer stores past transitions and samples them randomly, which breaks temporal correlation and improves data efficiency. These techniques make DQN suitable for learning in high-dimensional spaces and have been applied to traffic signal control. However, DQN is limited to discrete actions and can still suffer from overestimation. To improve performance, several extensions have been proposed. Double DQN reduces overestimation by separating action selection and evaluation [22]. Dueling DQN separates state value and action advantage to enhance learning [23]. Rainbow DQN combines multiple techniques, including Double DQN, Dueling networks, prioritized replay, noisy exploration, and multi-step learning [24], and achieves strong results in adaptive signal control tasks.

### 2.2.2 Actor-Critic and Policy Gradient Methods

In Actor-Critic method, **policy network (actor)** decides action depends on the current state and **value network (critic)** estimates the state-value  $V(s)$  or the

action-value  $Q(s, a)$  to guide the policy update. And depending on the policy and data usage, actor-critic methods can be classified into two main categories:

- **On-Policy Stochastic Actor-Critic:** These methods learn a stochastic policy directly from the most recent interactions. Examples include Asynchronous Advantage Actor-Critic (A3C) and Proximal Policy Optimization (PPO).
- **Off-Policy Deterministic Actor-Critic:** These methods use a deterministic policy and can reuse historical data. A typical example is Deep Deterministic Policy Gradient (DDPG).

**Asynchronous Advantage Actor-Critic (A3C).** [25–28] is a on-policy actor-critic method that accelerates training by deploying multiple parallel agents interacting with their own environments. Each agent updates a shared global network asynchronously to enhance exploration and reduce sample correlation. A3C uses the advantage function  $A(s, a) = Q(s, a) - V(s)$  to reduce variance in policy gradient estimation. However, due to the lack of shared memory among agents and its sensitivity to exploration schedules, A3C often has unstable problem.

**Deep Deterministic Policy Gradient (DDPG).** DDPG [29–32] is an off-policy algorithm designed for continuous control tasks. It employs a deterministic policy and a critic network to estimate. DDPG incorporates several techniques such as target networks and experience replay. This design enables the agent to reuse past interactions and learn more efficiently. While DDPG is effective for precise control like adjusting green phase durations. It is still prone to Q-value overestimation and instability.

**Proximal Policy Optimization (PPO).** PPO [33–36] is an on-policy stochastic actor-critic method that addresses training instability through a clipped surrogate objective. By clipping the probability ratio between new and old policies. This mechanism enhances learning stability without requiring a complex trust region optimization. PPO supports both discrete and continuous actions, making it

suitable for tasks such as selecting signal phases or deciding green time extensions. However, it remains sample-inefficient and sensitive to the clipping hyperparameter.

### 2.2.3 Recurrent Policies in Traffic Signal Control

In most deep reinforcement learning algorithms used in traffic signal control. What the agent sees is incomplete, for example, key traffic conditions like upstream congestion or residual queues might not be directly observable. This kind of uncertainty leads to what’s called a partially observable Markov decision process (POMDP), where the agent needs to infer hidden patterns based on limited information. Standard feedforward RL models, such as DQN or PPO, aren’t designed to handle this. To address this limitation, **Recurrent Neural Networks (RNNs)** have been incorporated into reinforcement learning policies. These models allow the agent to implicitly remember and reason over past observations. When combined with actor-critic frameworks, this leads to **Recurrent Proximal Policy Optimization (R-PPO)** [37, 38], an extension of PPO that integrates LSTM layers into both the actor and the critic networks. R-PPO updates its policy based on not only the current observation but also the accumulated memory of prior states. This is valuable in traffic environments such as roundabouts, where merging pressure and congestion levels evolve gradually over time.

Table 1: Comparison of Deep RL Methods for Traffic Signal Control.

| Method         | Learning Mode | Policy                 | Limitations                        |
|----------------|---------------|------------------------|------------------------------------|
| DQN [18–24]    | Off-policy    | Q-network              | Overestimation;<br>Discrete action |
| [25–28]        | On-policy     | Advantage actor-critic | High variance; no<br>shared memory |
| DDPG [29–32]   | Off-policy    | Deterministic gradient | Unstable with<br>sparse rewards    |
| PPO [33–36]    | On-policy     | Clipped objective      | clip range sensitive               |
| R-PPO [37, 38] | On-policy     | Clipped + LSTM         | slower convergence                 |

## 3. Proposed Method

This chapter introduces the proposed traffic signal control framework, which combines a traffic simulation platform with a memory-enhanced deep reinforcement learning (DRL) agent. The chapter consists of the following sections:

- Section 3.1 provides an overview of the DRL-based control loop and explains the interaction between agent and environment.
- Section 3.2 presents the overall system architecture and how the traffic simulator is integrated with the DRL training process.
- Section 3.3 explains the design of DRL components-observation, action and reward.
- Section 3.4 describes the core learning algorithm - Recurrent Proximal Policy Optimization (RPPO).

### 3.1 Reinforcement Learning for Traffic Signals

Figure 1 illustrates the fundamental closed-loop interaction cycle of traffic signal control. The reinforcement learning agent continuously interacts with the environment in three sequential steps:

1. **Observation:** The agent receives current traffic state information.
2. **Action:** Based on the observation the agent selects an action, typically switch the traffic light phase or extend the current green.
3. **Reward:** After the action is applied, the environment transitions and returns a reward signal that evaluate action performance. Then the agent updates its policy based on this reward.

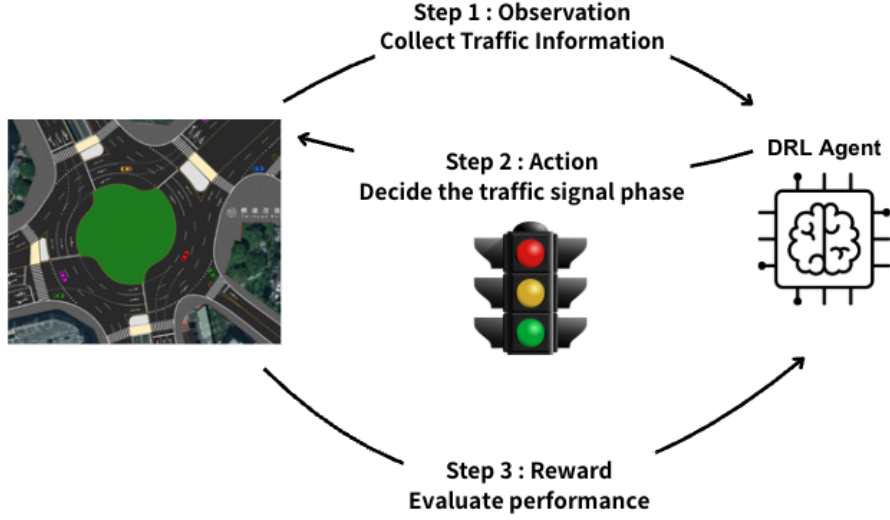


Figure 1: Conceptual loop of DRL-based traffic signal control.

## 3.2 System Architecture

The proposed system combines the traffic simulator SUMO (Simulation of Urban Mobility) [13] with a reinforcement learning agent trained using recurrent proximal policy optimization (R-PPO). SUMO provides a detailed simulation of urban traffic, and the R-PPO agent learns how to control traffic signals by interacting with the simulated environment.

Figure 2 shows the system structure, which includes the following parts:

- Traffic Simulator (SUMO): Simulates vehicle flows and traffic signals.
- TraCI API: Enables real-time control and data exchange with SUMO.
- Environment Module: Defines observations, actions, and rewards for training.
- R-PPO Agent: An LSTM-deep reinforcement learning model for signal control.
- Rollout Buffer: Stores trajectories for policy updates.
- Signal Controller: Executes agent actions in SUMO.



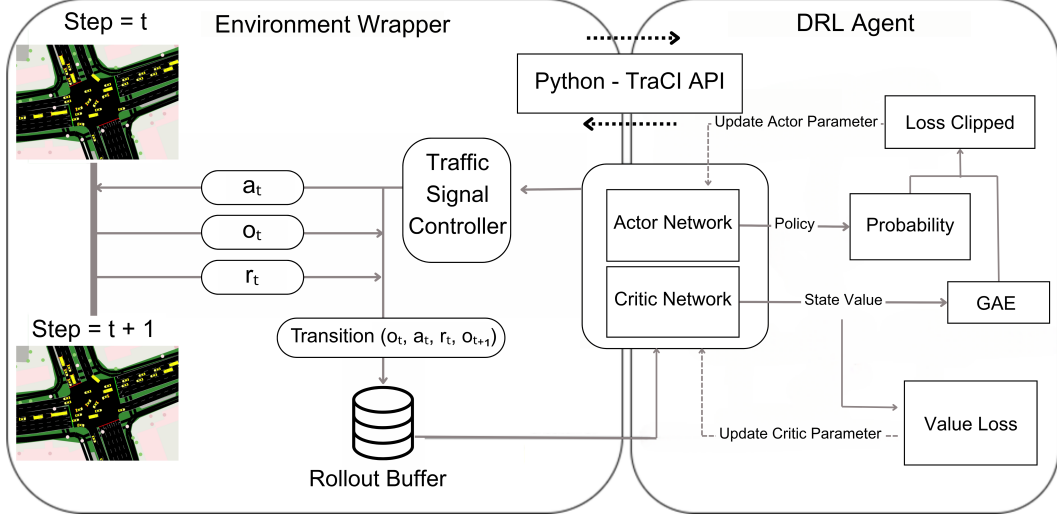


Figure 2: Proposed system architecture.

The system operates in the following loop:

1. The SUMO simulator advances one step ( $t \rightarrow t + 1$ ), updating vehicle and signal states.
2. The environment module collects traffic features and outputs the observation vector  $o_t$ .
3. The R-PPO agent uses the observation history  $o_0, \dots, o_t$  to generate the action  $a_t$ , following its stochastic policy  $\pi_\theta(a_t|o_{\leq t})$ .
4. The signal controller applies  $a_t$  in SUMO via the TraCI API.
5. The environment transitions to the next state, producing  $o_{t+1}$  and the reward signal  $r_t$ , which reflects traffic performance.
6. The transition tuple  $(o_t, a_t, r_t, o_{t+1})$  is stored in the rollout buffer. After collecting a batch, the agent updates the policy  $\pi_\theta$  and value function  $V_\phi(o_t)$  using PPO with the advantage estimate  $A_t$  computed by Generalized Advantage Estimation (GAE).

Table 2: Notations used in the DRL training framework

| Symbol                       | Definition                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| $o_t$                        | Observation vector at timestep $t$ , extracted from the environment (e.g., lane occupancy, queue length, signal timer). |
| $a_t$                        | Action at timestep $t$ , representing a signal phase switch or a green time extension.                                  |
| $r_t$                        | Reward signal based on traffic performance metrics after executing $a_t$ .                                              |
| $o_{t+1}$                    | Observation after the environment transitions following action $a_t$ .                                                  |
| $(o_t, a_t, r_t, o_{t+1})$   | Transition tuple stored in the rollout buffer for policy learning.                                                      |
| $\{o_0, \dots, o_t\}$        | Sequence of historical observations used by the recurrent policy.                                                       |
| $\pi_\theta(a_t o_{\leq t})$ | The stochastic policy with parameters $\theta$ outputs a distribution over actions given past observations.             |
| $V_\phi(o_t)$                | Value function estimating expected return from $o_t$ , with parameters $\phi$ .                                         |
| $A_t$                        | Advantage estimate used in policy updates, computed via Generalized Advantage Estimation (GAE).                         |

### 3.3 Component Design

This section presents the detailed design of the observation space, action space, and reward structure adopted in the proposed Recurrent Proximal Policy Optimization (R-PPO) framework.

#### 3.3.1 Observation Space

At each simulation timestep  $t$ , the environment provides a normalized observation vector  $o_t \in \mathbb{R}^d$ , capturing both traffic conditions and signal-related information. The observation includes features from three categories:

**(1) Approach-Level Features** For each of the six incoming approaches, the following features are extracted:

- **Lane occupancy:**

$$o_{\text{occ}}^{(i)} = \frac{1}{|\mathcal{L}_i|} \sum_{l \in \mathcal{L}_i} \frac{\text{occ}(l)}{100} \quad (1)$$

where  $\text{occ}(l)$  is obtained via `traci.lane.getLastStepOccupancy()`.

- **Vehicle flow:**

$$o_{\text{flow}}^{(i)} = \min \left( 1.0, \frac{1}{|\mathcal{L}_i|} \sum_{l \in \mathcal{L}_i} \frac{n_l}{\rho_{\text{ref}}} \right) \quad (2)$$

where  $n_l$  denotes the number of vehicles on lane  $l$ , and  $\rho_{\text{ref}} = 10$  is the reference number for normalization.

- **Mean lane speed:**

$$o_{\text{speed}}^{(i)} = \frac{1}{|\mathcal{L}_i|} \sum_{l \in \mathcal{L}_i} \frac{v_l}{v_{\text{max}}} \quad (3)$$

where  $v_l$  is the mean speed on lane  $l$ , and  $v_{\text{max}} = 8.0$  m/s is the upper bound.

This part contributes  $6 \text{ approaches} \times 3 \text{ features} = 18$  dimensions.

**(2) Ring-Level Features** To capture overall traffic conditions within the roundabout, the following features are used:

- **Mean ring occupancy:** Average occupancy across all ring lanes:

$$o_{\text{ring-occ}} = \frac{1}{|\mathcal{R}|} \sum_{l \in \mathcal{R}} \frac{\text{occ}(l)}{100} \quad (4)$$

where  $\text{occ}(l)$  is obtained via `traci.lane.getLastStepOccupancy(l)`.

- **Ring speed percentiles:** Normalized 25th and 75th percentile vehicle speeds on the ring:

$$o_{\text{speed\_p25}} = \text{clip} \left( \frac{P_{25}}{v_{\text{max}}}, 0.0, 1.0 \right), \quad o_{\text{speed\_p75}} = \text{clip} \left( \frac{P_{75}}{v_{\text{max}}}, 0.0, 1.0 \right) \quad (5)$$

where  $P_{25}$  and  $P_{75}$  are computed using `traci.vehicle.getSpeed(veh_id)` on vehicles within the ring.

These features contribute 3 dimensions to the observation space.

**(3) Phase-Related Features** To inform the agent of the current signal status and timing, the following features are used:

- **Phase mask:** One-hot vector  $o_{\text{mask}} \in \{0, 1\}^{n_{\text{green}}}$  indicating the active green phase.
- **Timer ratio:** Elapsed time of the current phase:

$$o_{\text{timer}} = \text{clip} \left( \frac{t_{\text{curr}}}{T_{\text{max}}}, 0, 1 \right) \quad (6)$$

- **Episode progress:** Episode progress:

$$o_{\text{progress}} = \text{clip} \left( \frac{t}{T_{\text{end}}}, 0, 1 \right) \quad (7)$$

These features add  $n_{\text{green}} + 2$  dimensions to the observation space. With  $n_{\text{green}} = 3$ , this results in 5 dimensions.

**Final Observation Vector** The complete observation vector  $o_t$  is constructed by concatenating all normalized features:

$$s_t = \left[ \underbrace{s_{\text{occ}}^{(1)}, s_{\text{flow}}^{(1)}, s_{\text{speed}}^{(1)}, \dots, s_{\text{occ}}^{(6)}, s_{\text{flow}}^{(6)}, s_{\text{speed}}^{(6)}}_{\text{Approach-level (18D)}}, \underbrace{s_{\text{ring\_occ}}, s_{\text{speed\_p25}}, s_{\text{speed\_p75}}}_{\text{Ring-level (3D)}}, \underbrace{s_{\text{mask}}, s_{\text{timer}}, s_{\text{progress}}}_{\text{Phase-related (5D)}} \right] \quad (8)$$

This results in a 26-dimensional observation vector. The observation space integrates both local and global traffic information. Approach-level features capture congestion and flow at each entry arm, while ring-level features reflect overall roundabout dynamics. Phase-related features, including the phase mask and timer ratio, provide the agent with the current signal state and phase timing. This design enables the agent to make signal control decisions with awareness of immediate local traffic conditions and overall circulation within the roundabout.

### 3.3.2 Action Space

At each timestep, the agent selects a discrete action  $a_t = \phi_t$ , where  $\phi_t \in \{0, 1, 2\}$  denotes the index of the target green phase. This action indicates the agent's intention to switch to one of the three predefined green phases. But the final execution of the phase transition is still subject to several safety and operational constraints:

- **Minimum green duration (`min_hold` = 15 seconds):** A currently active green phase must remain active for at least 15 seconds before a switch to another phase is allowed. This prevents excessive phase flipping and ensures stable signal timing.
- **Maximum green duration (`max_hold` = 30 seconds):** If the current green phase reaches 30 seconds, a forced switch to a different phase is triggered, regardless of the agent's decision. This avoids starvation of other directions.
- **Yellow buffer interval = 3 seconds:** A fixed 3-second yellow buffer is inserted between any two green phases to ensure safety during phase transitions. During this buffer interval, no green light is active and the agent's action is temporarily ignored until the buffer completes.

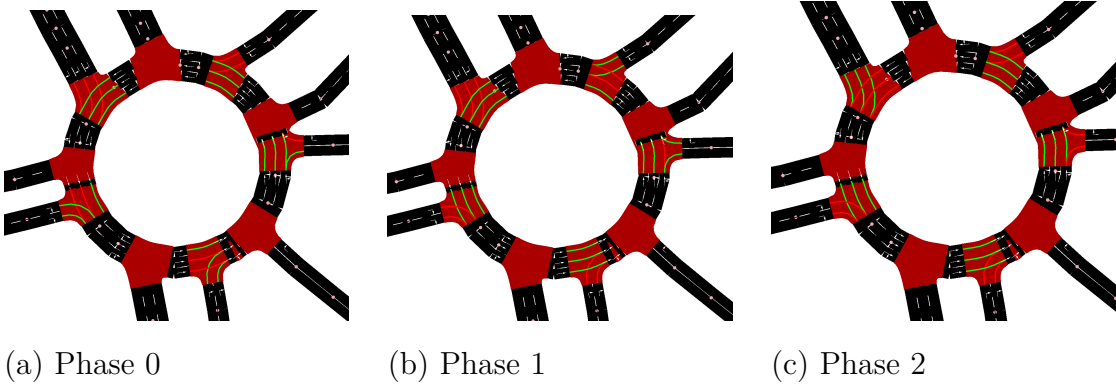


Figure 3: Index of the target green phase.

### 3.3.3 Reward Design

The reward  $r_t$  is made of several components. It is designed to improve traffic efficiency, encourage faster movement, and reduce unnecessary phase switching. The total reward is:

$$r_t = r_{\text{outflow}} + r_{\text{speed}} + r_{\text{slow}} + r_{\text{qmax}} + r_{\text{switch}} \quad (9)$$

Each part is explained below:

- **Ring Outflow reward** ( $r_{\text{outflow}}$ ): This reward provides a positive signal when a vehicle successfully exits the roundabout.

$$r_{\text{outflow}} = \text{clip}\left(\frac{n_{\text{exit}}}{10.0}, 0.0, 1.0\right) \quad (10)$$

where  $n_{\text{exit}}$  is the number of vehicles that transitioned from non-exit lanes to exit lanes in the current step.

- **Average speed reward** ( $r_{\text{speed}}$ ): This term increases when the average speed of vehicles is higher:

$$r_{\text{speed}} = \frac{1}{n_{\text{veh}}} \sum_{i=1}^{n_{\text{veh}}} \frac{v_i}{8.0} \quad (11)$$

where  $v_i$  is the speed of vehicle  $i$ , and 8.0 m/s is the reference maximum speed.

- **Slow ratio penalty** ( $r_{\text{slow}}$ ): This penalty increases when there are many vehicles moving slower than 2.0 m/s:

$$r_{\text{slow}} = -\frac{n_{\text{slow}}}{n_{\text{veh}}} \quad (12)$$

where  $n_{\text{slow}}$  is the number of slow vehicles, and  $n_{\text{veh}}$  is the total vehicles.

- **Queue max penalty** ( $r_{\text{qmax}}$ ): This term gives a negative reward when any approach has a very long queue:

$$r_{\text{qmax}} = -\tanh\left(\frac{q_{\text{max}}}{20}\right) \quad (13)$$

where  $q_{\text{max}}$  is the longest queue among all approaches.

- **Phase switch penalty** ( $r_{\text{switch}}$ ): This penalty is applied only when the traffic light phase is changed:

$$r_{\text{switch}} = \begin{cases} \alpha_t, & \text{if the phase switches at time } t \\ 0, & \text{otherwise} \end{cases} \quad (14)$$

The value of  $\alpha_t$  becomes more negative over time. It changes linearly from  $-0.2$  to  $-0.5$  during the first 150,000 training steps.

This reward function helps the agent learn to reduce congestion, keep vehicles moving, and avoid unnecessary switching. The lag-based reward  $r_{\text{wait}}$  gives more stable feedback compared to only using current waiting time.

**Dynamic normalization with VecNormalize** To enhance training stability, this study wraps the environment with **VecNormalize**, a utility provided by Stable-Baselines3 [39]. This module continuously estimates the mean and standard deviation for each observation feature and the reward, enabling dynamic normalization according to the Z-score standardization formula:

$$\hat{o}_t^{(i)} = \frac{o_t^{(i)} - \mu^{(i)}}{\sigma^{(i)} + \epsilon}, \quad \hat{r}_t = \frac{r_t - \mu_r}{\sigma_r + \epsilon} \quad (15)$$

Here,  $\mu^{(i)}$  and  $\sigma^{(i)}$  denote the running mean and standard deviation of the  $i$ -th observation dimension. The small constant  $\epsilon$  is added to prevent dividing by near-zero values. This computation is equivalent to Z-score normalization, which transforms input variables into a zero-mean, unit-variance form. This dynamic standardization provides several key benefits:

- It reduces gradient variance and stabilizes policy updates.
- It ensures consistent input scaling across time steps.

### 3.4 Recurrent Proximal Policy Optimization

This study applies Recurrent Proximal Policy Optimization (RPPO) to traffic signal control under sequential decision-making and partial observability. RPPO extends Proximal Policy Optimization (PPO) by adding a Long Short-Term Memory (LSTM) layer to the policy, allowing the agent to utilize past traffic information when making decisions. This memory-based design is suitable for urban traffic, where vehicle queues and delayed responses require temporally informed control. The RPPO policy architecture, including the observation encoder, recurrent memory, and dual output heads for action and value estimation, is shown in Figure 4.

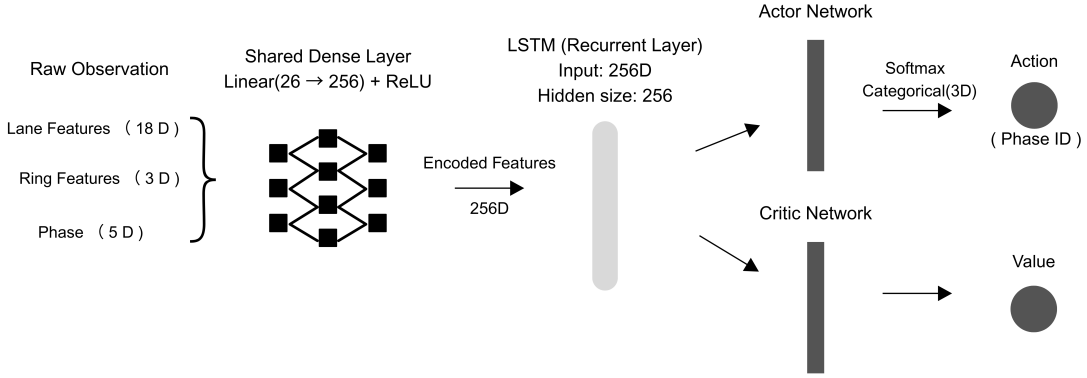


Figure 4: Architecture of the RPPO policy network with a shared encoder, LSTM memory, and actor-critic output heads.

#### 3.4.1 Policy Network Architecture

The RPPO policy network is designed to process sequential observations and output both control decisions and value predictions. It consists of four main components: observation encoder, LSTM-based memory module, actor head, and critic head.



**Observation Encoding** At each decision step  $t$ , the agent receives a normalized observation vector  $o_t \in \mathbb{R}^{26}$ , composed of three categories of traffic features:

- **Approach-level features (18D)**: normalized lane occupancy, vehicle inflow rate (per-lane count normalized by a reference value), and mean lane speed (normalized by maximum allowed speed) across six entry approaches.
- **Ring-level features (3D)**: average occupancy of the ring lanes and normalized 25th/75th percentiles of vehicle speed within the roundabout.
- **Phase-related features (5D)**: current phase mask (one-hot vector indicating the active green phase), phase timer ratio (elapsed time in the current phase normalized by maximum phase duration), and episode progress ratio (current simulation time normalized by episode duration).

These features are concatenated and passed through a multi-layer perceptron (MLP) to extract compact representations:

$$x_t = \text{MLP}(o_t), \quad x_t \in \mathbb{R}^{256}$$

**Recurrent Memory Module** To capture temporal dependencies, the output  $x_t$  from the observation encoder is passed into a Long Short-Term Memory (LSTM) module. The LSTM processes the current input along with its previous hidden and cell states to update the internal memory:

$$h_t, c_t = \text{LSTM}(x_t, h_{t-1}, c_{t-1})$$

Here,  $h_t \in \mathbb{R}^{256}$  and  $c_t \in \mathbb{R}^{256}$  denote the hidden and cell states at timestep  $t$ .

During training with  $N$  parallel environments, each environment instance  $n \in \{1, \dots, N\}$  maintains an independent recurrent state  $(h_t^{(n)}, c_t^{(n)})$ . And by using masking operation to prevent temporal information leakage between episodes.

**Actor Head** The actor head is responsible for selecting the next traffic signal phase. At each timestep  $t$ , the LSTM hidden state  $h_t \in \mathbb{R}^{256}$  is passed through a

linear transformation to produce unnormalized logits over the action space:

$$\text{logits}_\phi = \text{Linear}(h_t) \in \mathbb{R}^3$$

Each element in  $\text{logits}_\phi$  corresponds to one of the three available green phases, representing candidate signal control strategies for different traffic movements.

These logits are transformed into a probability distribution using the softmax function:

$$\pi(\phi_t \mid h_t) = \text{Softmax}(\text{logits}_\phi)$$

The agent then samples an action  $\phi_t \in \{0, 1, 2\}$  from this categorical distribution at every decision step. This phase index directly determines the next active green phase to be executed by the traffic signal controller.

**Critic Head** The critic head is used to estimate the expected cumulative return from the current state, which serves as a baseline for computing the advantage function. Similar to the actor, the critic operates on the LSTM hidden state  $h_t \in \mathbb{R}^{256}$  and outputs a scalar value prediction:

$$V(o_t, h_t) = \text{Linear}(h_t) \in \mathbb{R}$$

This value represents the expected future reward starting from timestep  $t$ , given the agent’s current observation  $o_t$  and its internal memory summarized by  $h_t$ .

Unlike the immediate reward  $r_t$ ,  $V(o_t, h_t)$  approximates the long-term return:

$$V(o_t, h_t) \approx \mathbb{E} \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k} \right]$$

where  $\gamma \in (0, 1)$  is the discount factor that determines the agent’s preference for short-term versus long-term rewards. Through training, the critic learns to predict this return based on interactions with the environment, even though it cannot directly observe the future. This value estimates are used to compute the

temporal-difference (TD) error at each timestep, defined as:

$$\delta_t = r_t + \gamma V(o_{t+1}, h_{t+1}) - V(o_t, h_t)$$

This error measures the difference between the predicted value and the observed outcome. A positive  $\delta_t$  indicates that the received reward and next state were better than expected, while a negative value means worse. The TD error serves two key purposes:

- It is the foundation for later computing the advantage function via Generalized Advantage Estimation (GAE).
- It is also used to train the critic itself by minimizing the squared prediction error during optimization.

### 3.4.2 Buffer, Advantage Estimation, and Loss Optimization

The training process of the RPPO agent consists of a structured pipeline that connects environment interaction, experience storage, advantage estimation, and policy optimization. During each rollout, the agent interacts with multiple parallel environments to collect sequences of experience tuples used to update the policy and value networks.

At every timestep, the agent receives an observation  $o_t$ , encodes it via an MLP, and processes it through the LSTM module to produce the updated hidden state  $h_t$ . Based on  $h_t$ , the actor selects an action  $\phi_t$  and the critic predicts the value estimate  $V(o_t, h_t)$ . After executing the action, the environment returns a reward  $r_t$  and the next observation  $o_{t+1}$ . These components, along with the LSTM hidden state and episode mask are recorded into a recurrent rollout buffer for training.

To enable gradient-based learning in the recurrent setting, the collected rollout is divided into fixed-length segments and trained using truncated backpropagation through time (BPTT). The critic estimates are used to compute temporal-difference errors and advantages using Generalized Advantage Estimation (GAE), while the actor is updated via the clipped surrogate loss of Proximal Policy Optimization.

**Experience Collection with Recurrent Rollout Buffer** Compared to standard replay buffers, this buffer keeps not only observations and actions, but also the LSTM hidden states, so the agent can learn from temporal dependencies.

At each timestep  $t$  in environment  $n$ , the buffer records the tuple:

$$(o_t^{(n)}, a_t^{(n)}, r_t^{(n)}, h_t^{(n)}, m_t^{(n)}, \pi_t^{(n)}, V_t^{(n)})$$

- $o_t^{(n)}$ : normalized observation vector,
- $a_t^{(n)}$ : selected action (phase index),
- $r_t^{(n)}$ : scalar reward,
- $h_t^{(n)}$ : LSTM hidden state before action,
- $m_t^{(n)} \in \{0, 1\}$ : mask indicating episode continuation,
- $\pi_t^{(n)}$ : action probability distribution,
- $V_t^{(n)}$ : predicted value before action.

These tuples are collected over a fixed number of steps, forming a batch of structured sequences. During training, the loss is computed over each segment, and gradients are propagated using truncated backpropagation through time (BPTT) to update the network parameters and reduce prediction error.

Each observation  $o_t$  is first processed by an MLP encoder, and the encoded result is sent into an LSTM unit to compute the hidden state  $h_t$ . The hidden state  $h_t$  is passed to the next timestep for sequence modeling. For training, BPTT is applied to calculate gradients only within the segment (e.g.,  $\frac{\partial h_1}{\partial h_0}$  and  $\frac{\partial h_{63}}{\partial h_{62}}$ ). These gradients are used to adjust the internal parameters of the LSTM module like the recurrent weights  $W_{hh}$ , input weights  $W_{xh}$ , and biases  $b$ . Updating these parameters enables the model to improve its ability to remember traffic patterns and temporal dependencies across time.

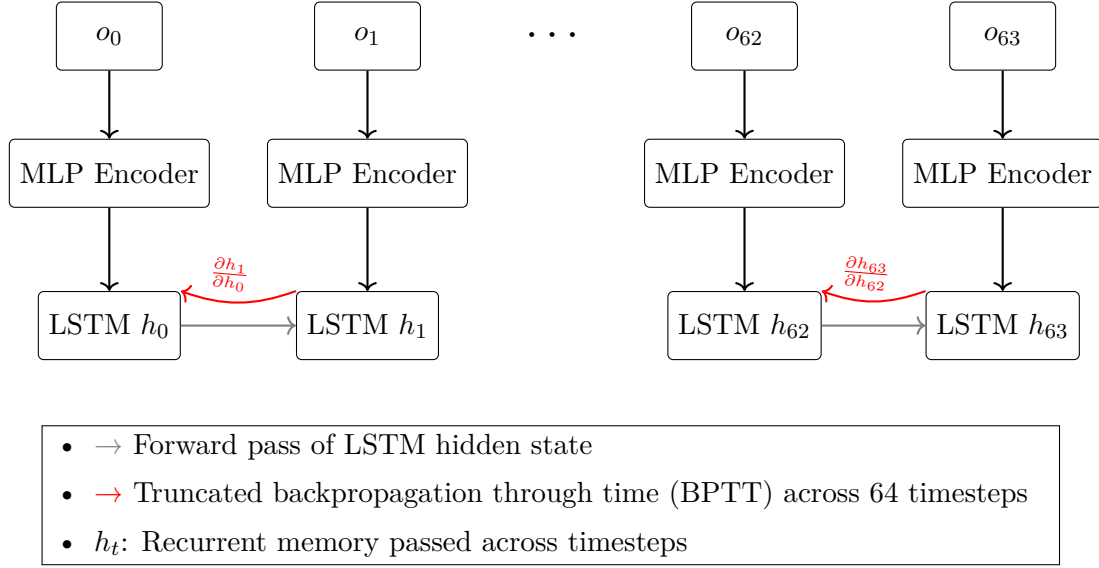


Figure 5: The forward and backward flow of hidden states in a 64-step rollout segment during Recurrent PPO training.

**Generalized Advantage Estimation (GAE)** The policy network is updated based on how much better or worse the chosen action performs compared to the expected outcome. This difference is called the *advantage*. GAE works by combining TD errors over multiple future steps, weighted by a discount factor and a smoothing parameter. GAE computes the advantage  $A_t$  by taking a weighted sum of future TD errors:

$$A_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots + (\gamma\lambda)^{T-t-1}\delta_{T-1}$$

Here,  $\lambda$  is a hyperparameter that controls how much the future TD errors contribute to the current advantage estimate. When  $\lambda = 0$ , only the immediate TD error is used, which results in low variance but high bias. In contrast, when  $\lambda = 1$  means all future TD errors are included which leads to low bias but high variance. The computed advantage  $A_t$  is used for two purposes. First, it is used in the PPO loss function to update action head. Second, it serves as a learning target for the value function.

**Clipped Policy Loss** To ensure stability during training, PPO limits how much the policy is allowed to change at each update step. This is done by computing a clipped surrogate objective:

$$\mathcal{L}_{\text{clip}}^{(t)} = \min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)$$

where  $r_t(\theta) = \frac{\pi_{\theta}(a_t|h_t)}{\pi_{\theta_{\text{old}}}(a_t|h_t)}$  is the probability ratio between the current and previous policies, and  $\epsilon$  is the clipping threshold. If  $A_t$  is positive, the objective encourages increasing the probability of the action; if  $A_t$  is negative, it encourages reducing the probability. The clipping function ensures that the policy is not updated too aggressively even when the advantage is large.

**Value Function Loss** The critic network is trained to minimize the prediction error between its estimated value and the actual return. This is expressed as:

$$\mathcal{L}_{\text{vf}}^{(t)} = \frac{1}{2} (V(o_t, h_t) - R_t)^2$$

where  $R_t$  is the training target for the value function. In practice, this return is computed as the sum of the GAE advantage and the baseline prediction, i.e.,  $R_t = A_t + V(o_t, h_t)$ . Accurate value estimates are essential because they reduce the variance of the advantage function.

**Entropy Bonus** To encourage exploration and prevent the policy from collapsing too early to a deterministic solution, PPO includes an entropy regularization term:

$$\mathcal{L}_{\text{entropy}}^{(t)} = -\beta \cdot \mathcal{H}[\pi(\cdot | h_t)]$$

Here,  $\mathcal{H}$  denotes the entropy of the action distribution and  $\beta$  is a coefficient that controls how strongly the model is encouraged to explore.

Based on the above architecture, the agent is trained using the Recurrent Proximal Policy Optimization framework. It collects trajectories from parallel environments and updates the policy using clipped surrogate loss, and trains the value network with GAE-based returns. The full procedure is shown in Algorithm 1.

---

**Algorithm 1:** Recurrent PPO Training with Parallel Environments
 

---

**1 Initialize:** Launch  $N$  parallel SUMO environments. For each episode, generate dynamic `rou.xml`. Initialize actor  $\mu_\theta(o_t, h_t^{\text{actor}})$ , critic  $V_\phi(o_t, h_t^{\text{critic}})$ , and rollout buffer  $\mathcal{B} = \{(o_t^{(n)}, a_t^{(n)}, r_t^{(n)}, o_{t+1}^{(n)}, d_t^{(n)}, h_t^{\text{actor},(n)}, h_t^{\text{critic},(n)})\}$ .

**2 for** episode  $e = 1$  to  $E$  **do**

**3     for** each environment  $n = 1$  to  $N$  in parallel **do**

**4         for** rollout step  $t = 1$  to  $T$  **do**

**5**Observe normalized state  $o_t^{(n)}$  and hidden state  $h_t^{(n)}$ .

**6**Select action  $a_t^{(n)} \sim \pi_\theta(a_t^{(n)} | o_t^{(n)}, h_t^{(n)})$ .

**7**Execute  $a_t^{(n)}$  in SUMO, receive reward  $r_t^{(n)}$ , next state  $o_{t+1}^{(n)}$ .

**8**Store  $(o_t^{(n)}, a_t^{(n)}, r_t^{(n)}, h_t^{(n)}, V_\phi(o_t^{(n)}))$  in the rollout buffer.

**9**Compute GAE advantages  $\hat{A}_t^{(n)}$  and returns  $\hat{R}_t^{(n)}$  for all  $n$  and  $t$ .

**10    for** each PPO update epoch **do**

**11**Update policy  $\theta$  by minimizing:

$$L^{\text{clip}} = \mathbb{E}_{n,t} [\min(r_t^{(n)}(\theta) \hat{A}_t^{(n)}, \text{clip}(r_t^{(n)}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{(n)})],$$

where

$$r_t^{(n)}(\theta) = \frac{\pi_\theta(a_t^{(n)} | o_t^{(n)}, h_t^{(n)})}{\pi_{\theta_{\text{old}}}(a_t^{(n)} | o_t^{(n)}, h_t^{(n)})}$$

**12**Update value function  $\phi$  by minimizing:

$$L^{\text{vf}} = \mathbb{E}_{n,t} [(V_\phi(o_t^{(n)}) - \hat{R}_t^{(n)})^2].$$

**13**Add entropy bonus:

$$L^{\text{ent}} = \mathbb{E}_{n,t} [\mathcal{H}(\pi_\theta(\cdot | o_t^{(n)}, h_t^{(n)}))].$$

**14**Perform gradient updates:

$$\theta \leftarrow \theta - \eta \nabla_\theta (L^{\text{clip}} - \beta L^{\text{ent}}), \quad \phi \leftarrow \phi - \eta_v \nabla_\phi L^{\text{vf}}$$

where  $\eta$  and  $\eta_v$  are learning rates, and  $\beta$  is the entropy coefficient.

**15 Output:** Trained Recurrent PPO policy  $\pi_\theta$ , value function  $V_\phi$ .

---

# 4. Experiments

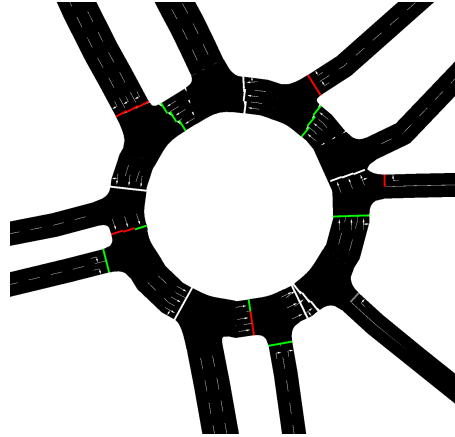
## 4.1 Simulation Setup

The evaluation is conducted on a simulation platform implemented in SUMO [13]. SUMO (Simulation of Urban Mobility) is a time-discrete, spatially continuous microscopic traffic simulation software that supports dynamic routing at intersections. It provides a visual graphical interface and supports multiple grid format inputs and flexible road network designs. This simulation provides a reliable platform for evaluating reinforcement learning-based signal control strategies.

**Network Topology** The Gongguan Roundabout consists of six arms with varying lane configurations and complex merging points. The base network is extracted from OpenStreetMap and further refined in SUMO’s NETEDIT to align with real-world road geometry and lane connections. Traffic engineering data from the Taipei City Government is used to configure junction priorities and detailed settings.



(a) Real-world layout



(b) SUMO simulation model

Figure 6: The real Gongguan Roundabout (a) and its SUMO simulation environment (b).



**Vehicle Composition and Traffic Demand** The simulation uses traffic demand data obtained from the Taipei City Government [40], including morning peak-hour vehicle entries and turning ratios. Figure 7 displays the per-approach vehicle inflow and turning behavior, while Table 3 shows the vehicle types, total volumes, and SUMO parameters.

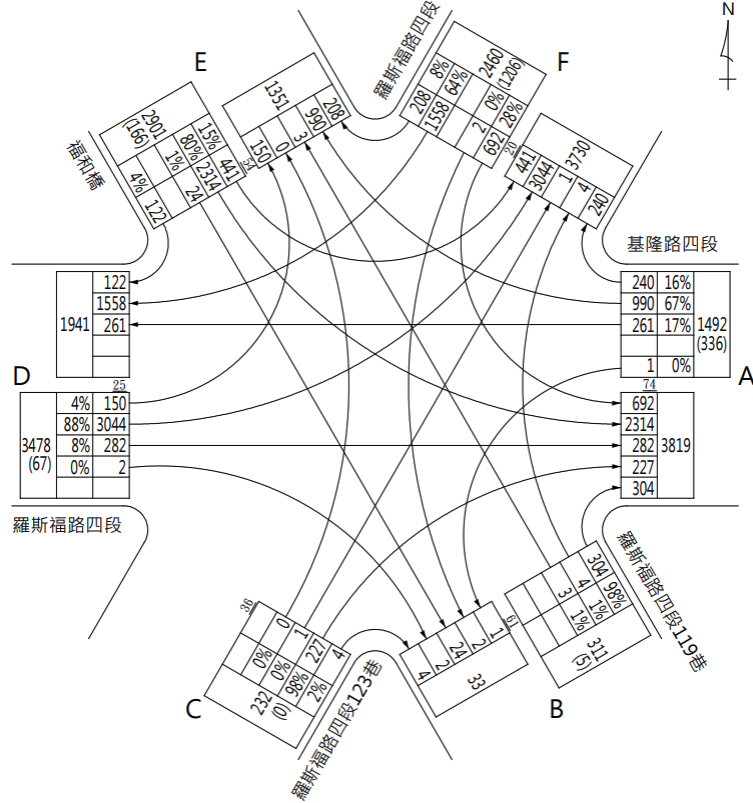


Figure 7: Entry volumes and turning ratios at the Gongguan Roundabout (data source: Taipei City Government).

Vehicles in the simulation are categorized into motorcycles, passenger cars, and heavy vehicles, each configured with specific parameters in SUMO. Vehicle insertion is managed using ‘rou.xml’ files with probabilistic distributions. Random seeds are varied across episodes to introduce traffic diversity while maintaining consistent overall demand ratios to ensure stable training conditions.

Table 3: Vehicle types, proportion and SUMO configuration parameters.

| Type          | Total Volume (Proportion) | SUMO Parameters           |            |
|---------------|---------------------------|---------------------------|------------|
| Motorcycle    | 8296 (76.3%)              | length=2.0,<br>decel=5.0  | accel=3.0, |
| Passenger Car | 2024 (18.6%)              | length=5.0,<br>decel=4.5  | accel=2.6, |
| Truck/Bus     | 554 (5.1%)                | length=12.0,<br>decel=3.5 | accel=1.2, |

## 4.2 Traffic Scenarios

To evaluate the proposed RPPO signal control strategy under realistic and challenging conditions, this study designs two traffic scenarios at the Gongguan Roundabout:

1. **High-Demand Scenario:** In this scenario, the overall traffic is increased to 130% of the baseline morning peak-hour traffic. Additionally, a noise of 10% is applied to the arrival rates for each route to introduce fluctuations in vehicle arrivals. This scenario evaluates the trained model’s ability to handle high and varying traffic demand while maintaining a stable signal control performance.
2. **Accident Scenario:** This scenario introduces a realistic incident by randomly blocking one lane. The closure stays for 15 minutes. This scenario tests the model’s adaptability under partial road blockage and its ability to maintain traffic flow during unexpected disruptions.

## 4.3 Baselines and Evaluation Metrics

To evaluate the effectiveness of the proposed RPPO-based signal control strategy, three baseline methods are used for comparison. All reinforcement learning methods use the same environment configuration, observation space, and reward structure, with the action space restricted to phase selection only for fair comparison.

**Fixed-Time Control** This classical baseline uses a fixed-cycle control strategy with manually configured green (30 seconds), yellow, and red durations for each phase based on average morning peak demand. The signal does not adapt to real-time traffic variations and serves as a lower-bound reference for non-adaptive control.

**Double DQN (DDQN)** Double Deep Q-Network is used as the value-based reinforcement learning baseline. The agent selects phases based on learned Q-values to maximize expected rewards, using double Q-learning to reduce overestimation bias during value updates.

**Proximal Policy Optimization (PPO)** PPO serves as the policy gradient-based baseline without recurrent structures. The agent learns a stochastic policy to select phases by optimizing a clipped surrogate objective to ensure stable updates. This comparison isolates the contribution of the recurrent memory structure in the proposed RPPO method.

To ensure fair comparison, all baseline methods are tested under the same environment settings, including network structure, simulation time, control interval, and vehicle types. The same random seeds are used in all runs to keep traffic variation and vehicle arrivals consistent. All methods also use the same observation, action and reward function design, so the performance differences come only from the algorithm itself. In terms of evaluating the proposed RPPO-based signal control method, this study uses four types of metrics. For traffic efficiency, the average speed reflects the overall traffic flow, and the average travel time shows how long vehicles take to pass through the roundabout. For congestion level, the average queue length measures the number of waiting vehicles. For learning performance, the average step reward, which is the episode reward divided by the number of steps, shows the learning progress and stability. To analyze control behavior, the average phase duration describes how frequent the agent switches traffic phases.

## 5. Results and Analysis

This chapter presents the experimental results of the proposed traffic signal control method. All experiments were conducted in same simulated roundabout intersection in Taipei City. And to ensure fair comparison, all methods use the same environment settings, observation space, and reward function. Each model was trained for 512,000 timesteps using identical random seeds. The objective is to evaluate the effectiveness of four control strategies: Recurrent Proximal Policy Optimization (RPPO), standard Proximal Policy Optimization (PPO), Double Deep Q-Network (DDQN), and a Fixed-Time Control baseline.

### 5.1 Training Performance and Stability

Figure 8 shows the average step reward curve. All models started with similar values near that reflecting heavy congestion. RPPO shows a consistent and significant increase, grow over zero after 150,000 timesteps and stabilizing between  $+0.25$  and  $+0.3$  by 200,000 steps. This trend indicates effective learning and convergence. PPO demonstrates moderate improvement but with fluctuations between  $-0.25$  and  $+0.1$ . DDQN remains between  $-0.9$  and  $-0.8$  throughout, suggesting difficulty in learning.

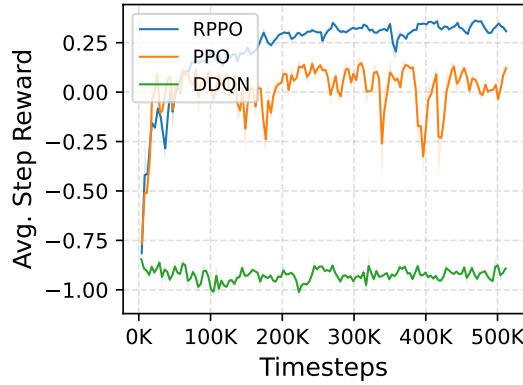


Figure 8: Training curve of average step reward for RPPO, PPO, and DDQN.

Figure 9 shows the progression of average vehicle speed during training. At the beginning, all three models start with similar performance around 3.0 to 3.2 m/s, indicating that the initial policies do not provide effective traffic control. As training progresses, RPPO shows a clear and steady improvement in traffic flow. Its average speed gradually increases and stabilizes at around 5.6 to 5.8 m/s, suggesting that the agent successfully learns to reduce congestion and maintain smooth vehicle movement. PPO also improves over time, although the trend is less stable. Its final speed reaches approximately 4.5 to 4.8 m/s, showing moderate success in managing flow but with some fluctuations. In contrast, DDQN maintains nearly flat performance throughout the training process. Its average speed remains close to the baseline, ending at only 3.5 to 3.6 m/s, which suggests that the value-based method struggles to find effective control policies under this setting. These results demonstrate that RPPO has stronger learning ability in optimizing continuous traffic performance, and PPO offers partial improvement and DDQN is less effective in this task.

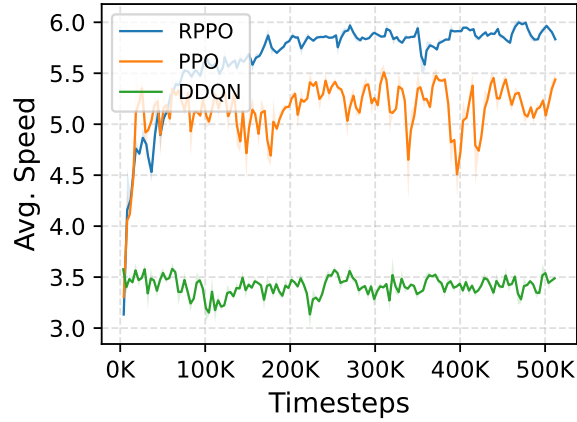


Figure 9: Training curve of average vehicle speed for RPPO, PPO, and DDQN.

Figure 10 presents the average queue length over time. At the start, all methods show similar initial queue lengths, approximately 30 vehicles, indicating comparable starting conditions. RPPO gradually reduces the queue length and reaches a stable

level below 15 vehicles after 200,000 timesteps. This shows that the memory-based policy is able to maintain consistent outflow and avoid long vehicle accumulation. PPO also reduces queue lengths, but the effect is less stable. It fluctuates between 25 and 30 vehicles, sometimes improving but occasionally returning to higher levels. It shows that the policy lacks consistency in decision-making. DDQN fails to improve the queue condition. Its average queue length stays above 40 vehicles throughout the training, indicating that the value-based strategy cannot adapt well to dynamic traffic pressure and cannot break out of suboptimal behavior. Overall, these results highlight the advantage of recurrent structures in managing long-term congestion, especially when queue buildup and dissipation require learning across multiple time steps.

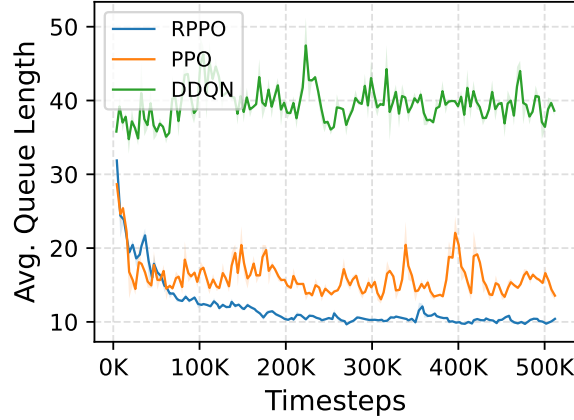


Figure 10: Training curve of average queue length for RPPO, PPO, and DDQN.

Figure 11 shows that RPPO maintains a high explained variance, stabilizing around 0.8–1.0 after 250K timesteps, while PPO stays lower at 0.3–0.7. This indicates that RPPO learns a more accurate value function, leading to better advantage estimation and more stable policy updates. In contrast, the fluctuating values in PPO suggest difficulties in fitting the return signal. Explained variance is not reported for DDQN, as it does not use a value baseline or advantage function.

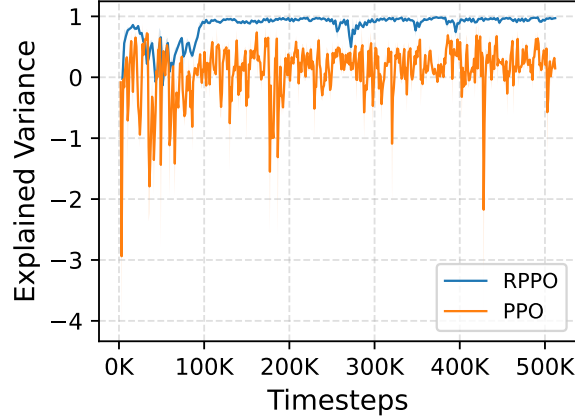


Figure 11: Explained variance of value function prediction during training.

## 5.2 Generalization Performance

To assess the generalization ability of different traffic signal control strategies, this section evaluates their performance under two challenging scenarios: high traffic demand and accident-induced lane blockage. All methods are tested using the same trained models from the baseline environment. The evaluation focuses on four key metrics: average step reward, average queue length, average vehicle speed, and average travel time. The results are presented in Tables 4 and 5.

### 5.2.1 High-Demand Scenario

Table 4 summarizes the performance of each method when the traffic demand is increased to 130% of the baseline, with a 10% random noise added to vehicle arrival rates. Among all methods, RPPO achieves the best overall performance. It has the highest average step reward ( $-0.13$ ), the lowest queue length (12.84 vehicles), the fastest average speed (5.18 m/s), and the shortest travel time (49.76 seconds). PPO and DDQN show moderate improvements over the fixed-time control baseline. In particular, the fixed-time method struggles to handle the increased demand, resulting in long queues (over 70 vehicles) and near-deadlock conditions. The

deadlock means vehicles inside the roundabout can not exit due to persistent congestion. These results demonstrate that RPPO is more effective in maintaining traffic throughput and reducing congestion under high-demand conditions.

Table 4: Performance under High-Demand Scenario

| Method      | Reward<br>(avg step)               | Queue<br>(veh)                     | Speed<br>(m/s)                    | Travel Time<br>(s)                 | Phase<br>(s)     |
|-------------|------------------------------------|------------------------------------|-----------------------------------|------------------------------------|------------------|
| Fixed-Time  | $-1.08 \pm 0.09$                   | $70.48 \pm 13.81$                  | $2.43 \pm 0.27$                   | $119.18 \pm 6.34$                  | $30.00 \pm 0.00$ |
| DDQN        | $-0.84 \pm 0.04$                   | $26.70 \pm 1.35$                   | $3.71 \pm 0.10$                   | $75.27 \pm 2.70$                   | $27.59 \pm 0.82$ |
| PPO         | $-0.63 \pm 0.04$                   | $23.42 \pm 1.05$                   | $4.18 \pm 0.08$                   | $66.57 \pm 2.10$                   | $21.90 \pm 0.27$ |
| <b>RPPO</b> | <b><math>-0.13 \pm 0.02</math></b> | <b><math>12.84 \pm 0.48</math></b> | <b><math>5.18 \pm 0.06</math></b> | <b><math>49.76 \pm 0.90</math></b> | $21.25 \pm 0.24$ |

### 5.2.2 Accident Scenario

In the accident scenario, a single lane is randomly blocked for 15 minutes to simulate a real-world disruption. As shown in Table 5, RPPO continues to outperform all other methods. It achieves the highest traffic efficiency with the lowest average queue length (13.40 vehicles), highest speed (5.17 m/s), and shortest travel time (55.90 seconds). The average step reward of RPPO ( $-0.17$ ) is also higher than PPO, DDQN, and Fixed-Time. While PPO performs reasonably well under this condition, its performance is slightly worse than RPPO across all metrics. In contrast, the fixed-time controller fails to adapt to the disruption, leading to severe congestion and excessive queue buildup (over 120 vehicles).

Table 5: Performance under Accident Scenario

| Method      | Reward<br>(avg step)               | Queue<br>(veh)                     | Speed<br>(m/s)                    | Travel Time<br>(s)                 | Phase<br>(s)     |
|-------------|------------------------------------|------------------------------------|-----------------------------------|------------------------------------|------------------|
| Fixed-Time  | $-1.37 \pm 0.14$                   | $122.70 \pm 33.00$                 | $1.61 \pm 0.34$                   | $94.80 \pm 39.90$                  | $30.00 \pm 0.00$ |
| DDQN        | $-1.25 \pm 0.09$                   | $42.60 \pm 8.50$                   | $2.87 \pm 0.28$                   | $108.70 \pm 20.40$                 | $26.02 \pm 1.18$ |
| PPO         | $-0.37 \pm 0.08$                   | $17.10 \pm 2.10$                   | $4.65 \pm 0.18$                   | $65.00 \pm 4.90$                   | $21.23 \pm 0.19$ |
| <b>RPPO</b> | <b><math>-0.17 \pm 0.11</math></b> | <b><math>13.40 \pm 2.70</math></b> | <b><math>5.17 \pm 0.25</math></b> | <b><math>55.90 \pm 5.30</math></b> | $20.60 \pm 0.13$ |



### 5.3 Discussion

**Handling of Delayed Congestion Effects:** Traffic congestion is often caused by actions that happened earlier, so current observations may not be enough to make the best decision. RPPO uses a recurrent structure (LSTM), which allows it to remember past traffic states and make better decisions in the present. For example, when a queue starts building up, RPPO can notice it earlier and take action before the situation becomes worse. PPO and DDQN do not have memory, so they only react to the current observation. This may explain why they were not as effective in dynamic or high-pressure conditions.

**Consistent Policy and Stable Behavior:** RPPO showed more stable behavior in terms of phase switching and traffic control. The values in the result tables show that RPPO had lower standard deviations in queue length and travel time. This suggests that the policy learned by RPPO is more consistent and does not change too much between different runs. PPO and DDQN sometimes switched phases too often or too early, which can interrupt the traffic flow and cause longer queues.

**Performance in Partially Observable Environments:** The accident scenario creates a more difficult environment because one lane is blocked for 15 minutes. This makes the traffic state partly unobservable, since the blocked lane can affect the whole system in a delayed way. In such cases, methods without memory cannot respond correctly. RPPO’s LSTM allows it to keep past traffic information, which helps the agent to understand the situation better. This may be one reason why RPPO still performed well even when the environment changed unexpectedly.

**Learning Difference Between Policy-Based and Value-Based Methods:** DDQN failed to improve during training. One possible reason is that PPO and RPPO are policy-based methods. They directly optimize the expected return by comparing actions using advantage estimates. This means that even if the reward difference between actions is small, the model can still learn useful updates. In contrast, DDQN is a value-based method and relies on estimating Q-values for

each action. If the reward is small and similar across different actions, the TD error becomes too weak to have updates. As a result, the Q-values do not change and the policy stays stuck in an early suboptimal solution. This may explain why DDQN performed slightly better than the fixed-time controller at the beginning but did not improve after that. It suggests that DDQN requires stronger reward signals or better exploration strategies to learn effectively in this environment.

**Effectiveness of Single-Step TD Error in Congested Traffic:** In a traffic environment with long-term heavy congestion, the traffic state often changes very slowly, and most vehicles remain stuck in queues. In such situations, it becomes difficult for recurrent models like RPPO to make use of historical information, because the past and current observations look very similar. As a result, learning from temporal patterns may not provide much benefit. In contrast, value-based methods such as DDQN rely on single-step temporal difference (TD) error to update their Q-values. This kind of immediate feedback helps the agent quickly identify which actions are slightly better, even when the overall traffic condition does not improve much. Although DDQN does not remember past traffic states, its greedy action selection based on TD error allows it to react directly to current rewards.

**Effect of Reward Design on Learning Behavior:** In this study, a phase-switch penalty was used to avoid too frequent signal changes. Changing the phase too often may reduce queues for a short time, but it also causes stop-and-go movement and more yellow signals, which reduce traffic efficiency. To solve this, the switch penalty was slowly increased during training. At the beginning, the agent could explore different phase actions more freely. Later, when the penalty became stronger, the agent learned to only switch phases when it is necessary. This helped the model reduce unnecessary switching and keep the signal behavior more stable.

# 6. Conclusions

## 6.1 Summary of Findings

This study applied a memory-based deep reinforcement learning method, Recurrent Proximal Policy Optimization (RPPO), to traffic signal control in a complex urban roundabout in Taipei. The target intersection, Gongguan Roundabout, presents unique challenges such as high motorcycle ratio, merging pressure, and frequent accidents. These characteristics make it difficult for traditional fixed-time or rule-based methods to adapt effectively. The proposed RPPO method was trained in a realistic SUMO-based environment and compared with three baselines. RPPO achieved more stable training performance, with better reward learning and value prediction. In testing, it outperformed the other methods in both high-demand and accident scenarios, maintaining higher speeds, shorter queues, and better adaptability. These results show that RPPO can generalize to difficult and dynamic conditions, thanks to its LSTM structure. Compared to baseline methods, RPPO better understand delayed congestion and adjusted signal timing. This study demonstrates that Deep reinforcement learning is a promising direction for future intelligent transportation systems.

## 6.2 Limitations

This study still has some limitations. First, all experiments are done in only one roundabout layout in Taipei City. The model is trained and tested in this fixed situation. If applying this method to other intersection type or bigger city network, observation features and control logic may need to redesign again. In this study, observation space and reward function are manually set by using common traffic indicators such as queue length and average speed. These handmade settings performed good in the current case, but might not perform well when road layout or traffic condition is different. If road structure changes, these features maybe become insufficient or less useful.

Second, the simulation environment assumes traffic data collected from vehicles and sensors are complete, accurate, and real-time available. But in real traffic systems, sensor delay, missing data, or communication failure often happen. These practical issues are not included in the simulation, which might negatively influence the real-world performance of the model. Also, although simulation includes four vehicle types. More complex and realistic driving behaviors, such as aggressive driving, illegal turns, or pedestrian randomly crossing the street, are not considered. These factors may increase the difficulty to manage traffic control in real-world situations.

Finally, the proposed RPPO method is based on deep reinforcement learning, which behaves like a black-box model. It is difficult to clearly know how decisions are made at each step. In this study, control actions can only be observed through basic statistics, like average phase duration and number of phase switches. For practical use, especially traffic control, it is important for human operators to understand or verify the model’s decision-making. Therefore, improving interpretability of the learned policy and testing it under more realistic conditions are important directions for future research.

## 6.3 Future Work

1. **Apply to larger networks:** The current model is trained and tested on only one specific roundabout layout. In the future, it is important to expand the approach on different types of intersections or larger road networks, such as multi-lane crossroads or connected multi-intersection systems.
2. **Learning-based observation and reward design:** In this study, the observation and reward functions are manually designed using traffic indicators such as occupancy, queue length, vehicle speed, and delay. These features are selected based on domain knowledge. While they work well in the current roundabout setup, they may not generalize to other types of intersections or cities. In future work, learning-based approaches can be applied to make both the observation and reward components more adaptive or transferable

across different traffic conditions.

- Learning observations from raw data: Manually selecting observation features has limitations, especially when dealing with large-scale or complex traffic environments. Instead of relying on fixed features, the agent can learn useful representations directly from raw data through neural networks. This approach allows the model to automatically discover patterns. For example:
  - Using traffic camera images or time-series sensor data (e.g., induction loops or radar) as input, and maybe applying convolutional neural networks (CNNs) to extract spatial patterns from the environment. For example, the model may learn to detect congestion shapes, vehicle density, or flow based on visual data.
  - Representing the road network as a graph structure, where intersections and road segments are modeled as nodes and edges. Graph neural networks (GNNs) can then be used to process traffic information across the network, such as queue length or delay at each node. This allows the agent to capture the spatial dependencies between different parts of the road network, and it is also important for multi-intersection systems.
- Learning reward functions automatically: Designing a good reward function is a critical step in reinforcement learning, as it directly affects the agent’s behavior. Future work can explore data-driven approaches to learn reward functions automatically, including:
  - Inverse reinforcement learning (IRL), which estimates the reward function by observing expert behavior. For example, if a human traffic engineer or a rule-based controller demonstrates good traffic management, the agent can infer what kind of outcomes the expert was trying to achieve, and use this as a reward signal.
  - Preference-based reward learning, where human users are asked to compare different behavior trajectories (e.g., two simulation clips)

and indicate which one is better. The agent can then use this feedback to refine its reward function, without requiring users to specify exact numerical values. This makes it easier to incorporate subjective preferences, such as comfort or perceived fairness.

- Adaptive reward weighting, which dynamically adjusts the importance of each reward component during training. For example, the agent might initially focus more on reducing queue length, but as learning progresses, gradually give more weight to smoothness or energy efficiency. This can be implemented using curriculum learning or meta-learning strategies.

3. **Handling imperfect and delayed information:** The current simulation assumes perfect, real-time traffic data. However, real-world systems often have sensor noise, missing data, or latency. Future work can explore policies under this situation. For example, POMDP formulations or noise-augmented training.
4. **Multi-agent extension:** This study adopts a single-agent controller for managing a single intersection. However, in large urban road networks, it needs to assign one agent to each intersection. Future research can explore a multi-agent framework, where each local agent operates independently while sharing information with neighboring intersections. Such decentralized coordination may improve overall scalability and enable more effective traffic control across complex city-wide systems.

# References

- [1] M. Tincani, “London congestion charge: The impact on air pollution,” *Population and Environment*, vol. 44, no. 2, pp. 238–267, 2022.
- [2] W. Tong, A. Hussain, X. B. Wang, and S. Maharjan, “Artificial intelligence for vehicle-to-everything: A survey,” *IEEE Access*, vol. 7, pp. 10823–10843, 2019.
- [3] A. Haydari and Y. Yilmaz, “Deep reinforcement learning for intelligent transportation systems: A survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 1, pp. 11–32, 2021.
- [4] Z. Xia, J. Wu, L. Wu, Y. Chen, J. Yang, and P. S. Yu, “A comprehensive survey of the key technologies and challenges surrounding vehicular ad hoc networks,” *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 12, no. 4, pp. 1–30, 2021.
- [5] Rhythm Engineering, “Insync adaptive traffic control shows initial safety benefits.” Presented at the ITE Western District Annual Meeting, 2014. Industry presentation.
- [6] P. Mirchandani and L. Head, “A real-time traffic signal control system: architecture, algorithms, and analysis,” *Transportation Research Part C: Emerging Technologies*, vol. 9, no. 6, pp. 415–432, 2001.
- [7] H. M. Abdelghaffar and H. A. Rakha, “A novel decentralized game-theoretic adaptive traffic signal controller: Large-scale testing,” *Sensors*, vol. 19, no. 10, p. 2282, 2019.
- [8] J. Gao, Y. Shen, J. Liu, M. Ito, and N. Shiratori, “Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network,” *arXiv preprint arXiv:1705.02755*, 2017.

- [9] M. Muresan, L. Fu, and G. Pan, “Adaptive traffic signal control with deep reinforcement learning: An exploratory investigation,” *arXiv preprint arXiv:1901.00960*, 2019.
- [10] S. Damadam, M. Zourbakhsh, R. Javidan, and A. Faroughi, “An intelligent iot based traffic light management system: Deep reinforcement learning,” *Smart Cities*, vol. 5, no. 4, pp. 1293–1311, 2022.
- [11] T.-E. Chou, “Reviewing taipei roundabout accident data and causes,” 2025.
- [12] W. E. Team, “Building a safe gongguan roundabout,” 2024.
- [13] P. Alvarez Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic traffic simulation using sumo,” in *The 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, IEEE, 2018.
- [14] Z. Liu, “A survey of intelligence methods in urban traffic signal control,” *International Journal of Computer Science and Network Security*, vol. 7, no. 7, pp. 105–112, 2007.
- [15] R. Abduljabbar, H. Dia, S. Liyanage, and S. A. Bagloee, “Applications of artificial intelligence in transport: An overview,” *Sustainability*, vol. 11, p. 189, Jan. 2019.
- [16] X. Liang, X. Du, G. Wang, and H. Zhu, “Deep reinforcement learning for traffic light control in intelligent transportation systems,” *arXiv preprint arXiv:2302.03669*, 2023.
- [17] Anonymous, “Adaptive traffic signal control based on dueling deep q-learning,” in *Proceedings of ACM Conference on Intelligent Transportation*, 2024.
- [18] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, and Veness, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.



- [19] H. Wei, G. Zheng, H. Yao, and Z. Li, “Intellilight: A reinforcement learning approach for intelligent traffic light control,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 2496–2505, ACM, 2018.
- [20] W. Genders and S. Razavi, “Using a deep reinforcement learning agent for traffic signal control,” 2016. arXiv preprint.
- [21] D. Ma, J. Ou, and Y. Ju, “Intelligent traffic signal control strategy based on rainbow algorithm,” in *2023 IEEE 8th International Conference on Intelligent Transportation Engineering (ICITE)*, 2023.
- [22] J. Gu, Y. Fang, Z. Sheng, and P. Wen, “Double deep q-network with a dual-agent for traffic signal control,” *Applied Sciences*, vol. 10, no. 5, p. 1622, 2020.
- [23] Y. Liang, L. Chen, M. Liu, and J. Wang, “Deep reinforcement learning for traffic light control in intelligent transportation systems,” *arXiv preprint*, vol. arXiv:2302.03669, 2023.
- [24] Y. Wang, F. Chen, X. Luo, and L. Zhang, “A signal timing optimization method based on rainbow dqn,” *Applied Sciences*, vol. 13, no. 3, p. 1101, 2023.
- [25] W. Genders and S. Razavi, “Evaluating reinforcement learning state representations for adaptive traffic signal control,” in *Proceedings of the International Conference on Computational Science (ICCS)*, vol. 130 of *Procedia Computer Science*, pp. 26–33, Elsevier, Jan. 2018.
- [26] U. P. Tewari, V. Bidawatka, V. Raveendran, V. Sudhakaran, S. Shreesail, and J. P. Kulkarni, “Intelligent coordination among multiple traffic intersections using multi-agent reinforcement learning,” in *arXiv preprint*, 2019.
- [27] S. Yang, B. Yang, H.-S. Wong, and Z. Kang, “Cooperative traffic signal control using multi-step return and off-policy asynchronous advantage actor-critic graph algorithm,” *Knowledge-Based Systems*, vol. 183, p. 104855, 2019.

- [28] H. Goel, Y. Zhang, M. Damani, and G. Sartoretti, “Sociallight: Distributed cooperation learning towards network-wide traffic signal control,” *arXiv preprint*, 2023.
- [29] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *arXiv preprint*, vol. arXiv:1509.02971, 2015.
- [30] Z. Li, C. Xu, and Wang, “A deep reinforcement learning approach for traffic signal control optimization,” *arXiv preprint*, vol. arXiv:2107.06115, 2021.
- [31] Z. Li, H. Yu, G. Zhang, and Q. Wang, “Network-wide traffic signal control optimization using a multi-agent deep reinforcement learning,” *Transportation Research Part C: Emerging Technologies*, vol. 129, p. 103208, 2021.
- [32] T. Wu, P. Zhou, K. Liu, Y. Yuan, X. Wang, H. Huang, and D. O. Wu, “Multi-agent deep reinforcement learning for urban traffic light control in vehicular networks,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 8, pp. 8243–8256, 2020.
- [33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint*, vol. arXiv:1707.06347, 2017.
- [34] L. Huang and X. Qu, “Improving traffic signal control operations using proximal policy optimization,” *IET Intelligent Transport Systems*, vol. 17, pp. 592–605, 2022.
- [35] Y. Zhu, H. Liu, and J. Wang, “Intelligent traffic light via policy-based deep reinforcement learning,” *arXiv preprint arXiv:2101.12345*, 2021.
- [36] M. Li, X. Pan, C. Liu, *et al.*, “Federated deep reinforcement learning-based urban traffic signal optimal control,” *Scientific Reports*, vol. 15, p. 11724, 2025.
- [37] D. Kakitahi, R. Xu, and M. Chen, “Adaptive traffic signal control based on multi-agent reinforcement learning,” *arXiv preprint arXiv:2504.09876*, 2025.

- [38] M. Pleines, M. Pallasch, F. Zimmer, and M. Preuss, “Generalization, mayhems and limits in recurrent proximal policy optimization,” 2022.
- [39] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [40] Taipei City Government, Department of Transportation, “2023 taipei city traffic volume and intersection characteristics survey,” technical report, Taipei City Government, 2023. Accessed July 2025.